1. Employee Management Analytics System

Problem Statement

Design an **Employee Management Analytics System**.

Create an `Employee` class with fields such as:

- employeeId
- name
- department
- salary
- experience
- activeStatus

Store multiple employee objects in a `List<Employee>`.

Using **Stream API**, write features to:

1. get all active employees
2. get employees whose salary is above a given threshold
3. count employees department-wise
4. find the highest-paid employee
5. get names of employees sorted by salary descending
6. group employees by department
7. get average salary department-wise

Concepts expected

- OOP class design
- `filter`
- `map`
- `sorted`
- `max`
- `groupingBy`
- `counting`
- `averagingInt` / `averagingDouble`

---

2. Student Result Processing System

Problem Statement

Design a **Student Result Processing System**.

Create a `Student` class with:

- rollNo
- name
- standard
- marks
- section

Store students in a collection.

Using **Stream API**, implement functionality to:

1. find students who passed based on a minimum marks rule
2. get top 3 students by marks
3. group students by section
4. count students section-wise
5. get average marks of each section
6. create a list of only student names in uppercase
7. find whether any student scored full marks

Concepts expected

- `filter`
- `limit`
- `sorted`
- `map`
- `groupingBy`
- `anyMatch`
- collectors

---

3. Online Order Processing System

Problem Statement

Design an **Online Order Processing System**.

Create an `Order` class with:

- orderId
- customerName
- category
- amount
- status

Store orders inside a `List<Order>`.

Using Stream API, perform:

1. get all completed orders
2. calculate total revenue from completed orders
3. group orders by category
4. find the order with maximum amount
5. count cancelled orders
6. generate a map of `orderId -> amount`
7. produce a sorted list of customer names based on order amount descending

Concepts expected

- `filter`
- `reduce`
- `map`
- `toMap`
- `groupingBy`
- `count`
- `max`

---

4. Library Book Inventory and Issue Tracker

Problem Statement

Design a **Library Book Inventory and Issue Tracker**.

Create a `Book` class with:

- bookId
- title
- author
- genre
- available
- price

Store books in a collection.

Using Stream API, implement:

1. get all available books
2. group books by genre
3. count books genre-wise
4. find the most expensive book
5. create a sorted list of titles by price ascending
6. check whether any unavailable book exists
7. create a set of all unique authors

Concepts expected

- `filter`
- `groupingBy`
- `counting`
- `map`
- `sorted`
- `anyMatch`
- `toSet`

---

5. Hospital Patient Record Analyzer

Problem Statement

Design a **Hospital Patient Record Analyzer**.

Create a `Patient` class with:

- patientId
- name
- age
- disease
- admitted
- billAmount

Store data in a `List<Patient>`.

Using Stream API, write code to:

1. get all admitted patients
2. group patients by disease
3. count admitted vs non-admitted patients
4. find patient with highest bill
5. calculate average bill amount
6. get names of patients above age 60
7. create a map of disease to list of patient names

Concepts expected

- `partitioningBy`
- `groupingBy`
- `mapping`
- `filter`
- `max`
- `map`
- `averagingDouble`

---

6. Product Inventory and Sales Analysis

Problem Statement

Design a **Product Inventory and Sales Analysis System**.

Create a `Product` class with:

- productId
- name
- category
- price
- stock
- soldUnits

Store products in a list.

Using Stream API, implement:

1. find out-of-stock products
2. group products by category
3. calculate total stock value (`price * stock`)
4. find product with highest sold units
5. list products sorted by sold units descending
6. create a map of product name to stock
7. check whether all products have stock greater than zero

Concepts expected

- `filter`
- `mapToDouble`
- `reduce` or `sum`
- `sorted`
- `toMap`
- `allMatch`

---

7. Transport Route Passenger System

Problem Statement

Design a **Transport Route Passenger System**.

Create a `Passenger` class with:

- passengerId
- name
- routeNo
- fare
- ticketType
- checkedIn

Store passengers in a collection.

Using Stream API, perform:

1. get all checked-in passengers
2. group passengers by route number
3. calculate total fare collected route-wise
4. get unique ticket types
5. sort passengers by fare descending
6. find whether any passenger has not checked in
7. create a list of passenger names for a particular route

Concepts expected

- `filter`
- `groupingBy`
- `mapping`
- `summingDouble`
- `distinct`
- `sorted`
- `anyMatch`

---

8. Course Enrollment and Performance Dashboard

Problem Statement

Design a **Course Enrollment and Performance Dashboard**.

Create a `Participant` class with:

- participantId
- name
- courseName
- score
- completed
- trainerName

Store participants in a `List<Participant>`.

Using Stream API, implement:

1. get all completed participants
2. group participants by course
3. get average score course-wise
4. find topper of each course
5. generate a set of trainers
6. get participants sorted by score descending
7. create a map of course to count of completed participants

Concepts expected

- `groupingBy`
- downstream collectors
- `averagingDouble`
- `maxBy`
- `mapping`
- `counting`
- `sorted`

---

9. Banking Transaction Statement Analyzer

Problem Statement

Design a **Banking Transaction Statement Analyzer**.

Create a `Transaction` class with:

- transactionId
- accountHolderName
- type (`DEPOSIT`, `WITHDRAWAL`)
- amount
- branch
- successful

Store transactions in a list.

Using Stream API, write code to:

1. filter all successful transactions
2. calculate total deposit amount
3. calculate total withdrawal amount
4. group transactions by branch
5. count failed transactions
6. create a map of branch to total successful transaction amount
7. find the largest transaction

Concepts expected

- `filter`
- `mapToDouble`
- `sum`
- `groupingBy`
- `count`
- `max`
- nested collectors

---

10. E-Commerce Customer Purchase Insights

Problem Statement

Design an **E-Commerce Customer Purchase Insights System**.

Create two classes:

`Customer`

- customerId
- name
- city

`Purchase`

- purchaseId
- customerId
- productName
- category
- amount

Store customers and purchases in collections.

Using Stream API, write features to:

1. get all purchases above a threshold amount
2. group purchases by category
3. calculate total purchase amount customer-wise
4. find the customer who spent the most
5. get unique product names
6. sort purchases by amount descending
7. for each city, count how many customers belong to it

Concepts expected

- OOP with multiple classes
- cross-collection reasoning
- `groupingBy`
- `toMap`
- `distinct`
- `sorted`
- aggregation logic